



ADVANCED TOPICS IN MICROSERVICES & CLOUD COMPUTING

Faculty of Technology - University of Sri Jayewardenepura

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science



MICROSERVICES ARCHITECTURE – OVERVIEW

- • Application is broken into small, independent services
- • Each service has its own database
- • Services communicate via APIs or messaging
- • Enables scalability, flexibility, and faster development



WHAT IS AN API GATEWAY?

- • Single entry point for all client requests
- • Routes requests to appropriate microservices
- • Handles cross-cutting concerns
- • Improves security and performance



GATEWAY RESPONSIBILITIES

- • Request routing
- • Authentication & Authorization
- • Load balancing
- • Rate limiting
- • Logging and monitoring

SPRING CLOUD GATEWAY

- • Lightweight and non-blocking
- • Built on Spring WebFlux
- • Supports filters and predicates
- • Works well with Eureka and Config Server

WHAT IS EUREKA SERVICE DISCOVERY?

Eureka Service Discovery is a **service registry** used in **microservices architecture** that allows services to **find and communicate with each other dynamically**, without hard-coding IP addresses or URLs. It is part of **Netflix OSS** and commonly used with **Spring Cloud**.

- • Service registry for microservices
- • Developed by Netflix
- • Helps services find each other dynamically
- • Avoids hard-coded service URLs

Why Service Discovery is Needed

- In microservices:
- Services are **deployed independently**
- IP addresses and ports **change dynamically**
- Services **scale up/down** frequently
- ➡ Hard-coding service locations is **not possible**
- **Eureka solves this problem.**

How Eureka Works

- **1 Eureka Server**
 - Central **service registry**
 - Maintains a list of all running services
 - Stores service name, IP, and port
- **2 Eureka Clients (Microservices)**
 - Register themselves with Eureka Server at startup
 - Send **heartbeat signals** to show they are alive
 - Query Eureka to discover other services

- **Communication Flow**

1. Microservice starts
2. Registers itself with **Eureka Server**
3. Client requests another service
4. Eureka returns available service instances
5. Request is load-balanced automatically

Key Features of Eureka

- ☒ Dynamic service registration
- ☒ Client-side load balancing
- ☒ Fault tolerance
- ☒ High availability (multiple Eureka servers)
- ☒ No hard-coded URLs

◆ Eureka vs Traditional Approach

Without Eureka

Hard-coded IPs

Poor scalability

Manual config

With Eureka

Dynamic discovery

Highly scalable

Automatic registration

- **Where Eureka is Used**
- API Gateway routing
- Inter-service communication
- Cloud-native applications
- Spring Cloud microservices

GATEWAY + EUREKA ARCHITECTURE

- • Clients call API Gateway
- • Gateway uses Eureka to locate services
- • Microservices communicate without knowing IPs
- • Enables dynamic scaling

CI/CD – CONTINUOUS INTEGRATION & DEPLOYMENT

- • CI: Code integration and testing frequently
- • CD: Automatic deployment to environments
- • Reduces manual errors
- • Speeds up delivery
- **CI/CD** is a software development practice that **automates building, testing, and deploying applications**, especially important in **microservices and cloud computing**.

What is Continuous Integration (CI)?

Continuous Integration (CI) means:

- Developers **frequently merge code** into a shared repository
- Each commit is **automatically built and tested**

◆ **CI Key Goals**

- Detect bugs early
- Improve code quality
- Reduce integration problems

◆ **CI Process**

1. Developer commits code to Git
2. CI tool triggers automatically
3. Code is built (Maven/Gradle)
4. Unit tests run
5. Build status reported

What is Continuous Deployment (CD)?

Continuous Deployment (CD) means:

- Automatically **deploying tested code** to servers or cloud
- No manual intervention (or minimal approval)

Sometimes CD is called **Continuous Delivery** (manual approval before production)

CI/CD Pipeline Stages

- | |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |

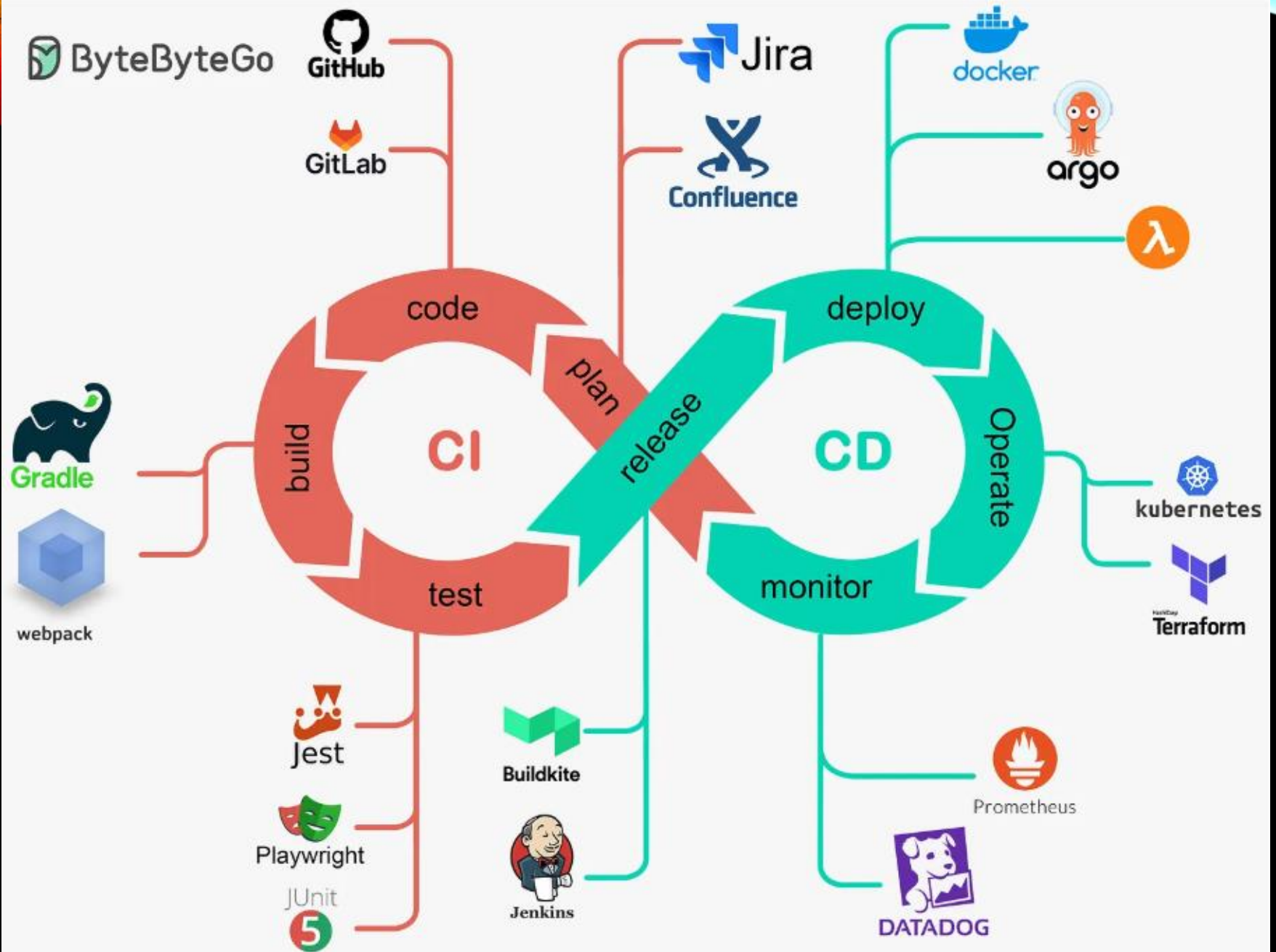
Source Code – GitHub / GitLab
- | |
|---|
| 2 |
|---|

Build – Compile code
- | |
|---|
| 3 |
|---|

Test – Unit & integration tests
- | |
|---|
| 4 |
|---|

Package – JAR / Docker image
- | |
|---|
| 5 |
|---|

Deploy – Dev / Test / Production



POPULAR CI/CD TOOLS

- • GitHub Actions
- • Jenkins
- • GitLab CI/CD
- • Azure DevOps
- • AWS CodePipeline

REMOTE SOURCE CODE MANAGEMENT

- **Remote Source Code Management** is the practice of **storing, managing, and maintaining source code in a remote (centralized or distributed) repository**, allowing multiple developers to collaborate efficiently.
- It is mainly done using **Version Control Systems (VCS)** like **Git**.



Why Remote Source Code Management is Needed

- Multiple developers work on the same project
- Code needs to be backed up securely
- Changes must be tracked
- Easy collaboration from different locations

- **How Remote Source Code Management Works**

1. Code is stored in a **remote repository** (cloud/server)
2. Developers **clone** the repository locally
3. Changes are made locally
4. Changes are **committed**
5. Code is **pushed** back to remote repository
6. Other developers **pull** latest changes

POPULAR SCM PLATFORMS

- • GitHub
- • GitLab
- • Bitbucket
- • Azure Repos

◆ Common Git Commands

```
bash
```

```
git clone <repo-url>  
git add .  
git commit -m "message"  
git push origin main  
git pull
```



Key Features

- Version control
- Branching & merging
- Access control
- Collaboration & code review
- Issue tracking (in platforms like GitHub)

Role in CI/CD

- Code commit triggers CI pipeline
- Ensures latest code is always built and tested
- Enables automation

◆ Centralized vs Distributed SCM

Feature	Centralized (SVN)	Distributed (Git)
Repository	Single central repo	Multiple local repos
Offline work	✗ No	✓ Yes
Performance	Slower	Faster
Popularity	Less used	Industry standard



Benefits

- Safe and secure code storage
- Easy collaboration
- Change history tracking
- Backup & recovery

DISTRIBUTED MESSAGING SYSTEMS

- A **Distributed Messaging System** is a communication mechanism that allows **different services or applications to exchange messages asynchronously** across distributed environments such as **microservices and cloud platforms**.
- Instead of calling each other directly, services **send messages through a message broker or event stream**.

Why Distributed Messaging is Needed

- Microservices should be **loosely coupled**
- Systems must be **scalable**
- Services should not fail if another service is down
- Support **asynchronous communication**

- **How Distributed Messaging Works**

1. **Producer** sends a message/event

2. Message is stored in a **broker**

3. **Consumer** receives and processes the message

4. Producer and consumer are **independent**

Key Components

- **Producer** – Sends messages
- **Message Broker** – Stores and routes messages
- **Consumer** – Receives messages
- **Topic / Queue** – Message channel

Types of Messaging Models

1 Point-to-Point (Queue)

- One message → one consumer
- Used for task processing
- Example: **RabbitMQ Queue**

2 Publish–Subscribe (Topic)

- One message → multiple consumers
- Used for event notifications
- Example: **Kafka Topics**



◆ Popular Distributed Messaging Systems

System	Use Case
Apache Kafka	High-throughput, event streaming
RabbitMQ	Reliable message queuing
ActiveMQ	Enterprise messaging
AWS SQS	Cloud-managed queue

◆ Messaging vs REST Communication

REST API

Synchronous

Tight coupling

Immediate response

Less scalable

Messaging

Asynchronous

Loose coupling

Event-based

Highly scalable

Role in Microservices

- Event-driven architecture
- Improves fault tolerance
- Enables real-time processing
- Supports scalability

Example Use Case

Order Service sends an event →

Payment Service processes payment →

Notification Service sends email/SMS

(All services work independently)

Advantages

- High reliability
- Loose coupling
- Scalability
- Better performance

Challenges

- Message ordering
- Data consistency
- Monitoring complexity

POPULAR MESSAGING SYSTEMS

- • Apache Kafka – high throughput
- • RabbitMQ – message broker
- • ActiveMQ – enterprise messaging
- • AWS SQS – cloud managed queue



MESSAGING IN MICROSERVICES

- Event-driven architecture
- Better fault tolerance
- Supports real-time processing
- Improves system performance



SUMMARY

- Gateway handles routing and security
- Eureka enables service discovery
- CI/CD automates build & deployment
- SCM manages source code
- Messaging systems enable scalability

- 
- Thank you